

Proyecto Integrador de Sistemas de Recuperación de Información

Sistema de Búsqueda Musical con RAG y Embeddings Semánticos

Diego Hernandez Rodriguez C311, Fabian A. Almeida Martinez C311

Mayo 2026

Índice

1. Dominio Temático Seleccionado	3
1.1. Descripción del Dominio	3
1.2. Justificación del Dominio	3
2. Modelo de Recuperación de Información	4
2.1. Modelo Seleccionado: LSI + BM25 Híbrido con Mejora Semántica	4
2.2. Justificación de la Selección	4
2.2.1. LSI (Latent Semantic Indexing) - Componente Principal	4
2.2.2. BM25 (Best Matching 25) - Componente Secundario	4
2.2.3. Coincidencia Léxica - Componente de Apoyo	5
2.3. Arquitectura del Modelo de Recuperación	5
2.4. Fórmula de Puntuación Detallada	5
2.4.1. Componente Léxico	5
2.4.2. Componente Semántico	6
2.4.3. Puntuación Final según Tipo de Query	6
3. Fuentes Bibliográficas	7
4. Estadísticas del Corpus Recopilado	8
5. Arquitectura del Sistema	8
5.1. Flujo de Datos Completo	8
6. Módulos Implementados	9
6.1. Módulos Obligatorios	9
6.2. Módulos Opcionales Implementados	9
7. Manual de Usuario	10
7.1. Inicio del Sistema	10
7.2. Interfaz de Usuario	10
7.3. Tipos de Consultas Soportadas	10

8. Estrategia de Posicionamiento	11
8.1. Factores de Ranking	11
8.2. Justificación de la Estrategia	11
9. Opinión Crítica	12
9.1. Bondades del Sistema	12
9.2. Insuficiencias y Limitaciones	12
9.3. Trabajo Futuro	12
10.Despliegue con Docker	13

1. Dominio Temático Seleccionado

1.1. Descripción del Dominio

El sistema se ha diseñado específicamente para el dominio de **música y canciones**, abarcando:

Tipo de contenido	Descripción	Cantidad
Canciones	Títulos, artistas, álbumes	10K
Letras completas	Texto completo en múltiples idiomas	10K
Géneros musicales	Etiquetas de clasificación	20 categorías
Tags	Etiquetas descriptivas	50+ etiquetas
Metadatos adicionales	URLs, fechas, etc.	Variable

1.2. Justificación del Dominio

El dominio musical fue seleccionado por las siguientes razones:

1. **Riqueza semántica:** Las letras de canciones contienen lenguaje figurativo, emociones y metáforas, permitiendo aplicar técnicas avanzadas de NLP y embeddings semánticos.
2. **Naturaleza multilingüe:** Canciones en español, inglés, francés, portugués, italiano, etc., desafiando al sistema con procesamiento multilingüe.
3. **Aplicabilidad de RAG:** La generación aumentada por recuperación permite responder preguntas como “canciones tristes sobre amor” basándose en el contenido semántico, no solo coincidencias literales.
4. **Interés del usuario:** La búsqueda musical es una tarea común que combina necesidades exactas (“Bohemian Rhapsody”) con necesidades exploratorias (“canciones para llorar”).
5. **Disponibilidad de datos:** Existencia de datasets públicos estructurados (CSV con metadatos + letras).

2. Modelo de Recuperación de Información

2.1. Modelo Seleccionado: LSI + BM25 Híbrido con Mejora Semántica

El sistema implementa un **modelo híbrido de recuperación** que combina tres enfoques complementarios:

Componente	Modelo base	Peso en puntuación
LSI (Latent Semantic Indexing)	Modelo Vectorial Generalizado	40 %
BM25	Modelo Probabilístico de Lenguaje	30 %
Coincidencia léxica	Modelo Booleano Extendido	30 %

2.2. Justificación de la Selección

2.2.1. LSI (Latent Semantic Indexing) - Componente Principal

Fundamento: Deerwester et al. (1990) propusieron LSI como una técnica que utiliza descomposición en valores singulares (SVD) para descubrir relaciones semánticas latentes entre términos y documentos.

Justificación para dominio musical:

- Las letras de canciones usan sinónimos y metáforas (“heart”, “corazón”, “coeur” expresan amor)
- LSI captura estas relaciones semánticas que BM25 puro no detecta
- Permite encontrar canciones por tema (“ruptura amorosa”) sin palabras exactas

Implementación:

```
# TruncatedSVD de sklearn para reducir dimensionalidad
self.svd = TruncatedSVD(n_components=128, random_state=42)
X_reduced = self.svd.fit_transform(X_tfidf)
self.document_embeddings = normalize(X_reduced)
```

2.2.2. BM25 (Best Matching 25) - Componente Secundario

Fundamento: Robertson & Zaragoza (2009) desarrollaron BM25 como un modelo probabilístico de ranking que normaliza por longitud de documento.

Justificación:

- Maneja documentos de longitud variable (canciones de 100 a 5000 palabras)
- Parámetros $k_1=1.5$, $b=0.75$ optimizados para textos musicales
- Excelente para búsquedas exactas de frases en letras

2.2.3. Coincidencia Léxica - Componente de Apoyo

Fundamento: Modelo Booleano Extendido (Salton et al., 1975) con ponderación por campo.

Justificación:

- El título y artista son campos de alta importancia
- Búsquedas como “Bohemian Rhapsody Queen” deben priorizar coincidencia exacta
- Bonificaciones específicas mejoran experiencia de usuario

2.3. Arquitectura del Modelo de Recuperación

CONSULTA DEL USUARIO: "canciones tristes sobre amor"

PROCESAMIENTO DE LA CONSULTA

- Detección de idioma (es/en/fr/pt/it)
- Normalización de texto (acentos → ASCII)
- Tokenización y eliminación de stopwords
- Stemming básico por idioma

LSI	BM25	LÉXICO
(40%)	(30%)	(30%)

PUNTUACIÓN HÍBRIDA FINAL

$\text{score} = (\text{semántica} \times 40) + (\text{BM25_norm} \times 30) + (\text{léxico} \times 30)$

RANKING Y PRESENTACIÓN

2.4. Fórmula de Puntuación Detallada

2.4.1. Componente Léxico

```
score_lexico = 0
# Coincidencia exacta
if query == titulo:      score_lexico += 15.0
elif query == artista:   score_lexico += 12.0
elif query in titulo:    score_lexico += 8.0
elif query in artista:   score_lexico += 6.0

# Jaccard similarity
```

```

jaccard_titulo = |Q      T| / |Q      T|
score_lexico += jaccard_titulo      8.0
jaccard_artista = |Q      A| / |Q      A|
score_lexico += jaccard_artista      6.0

# BM25 normalizado y bonus inicial
BM25_norm = min(BM25_raw / 10.0, 5.0)
if titulo.startswith(primer_palabra): score_lexico += 2.0

```

2.4.2. Componente Semántico

```

score_semantico = cosine_similarity(emb_query, emb_doc) # [0,1]

# Factor multiplicativo adaptativo
if score_semantico < 0.3:
    factor = 0.1 + (score_semantico / 0.3)      0.4
elif score_semantico < 0.7:
    factor = 0.5 + ((score_semantico - 0.3) / 0.4)      0.5
else:
    factor = 1.0 + ((score_semantico - 0.7) / 0.3)      1.0

```

2.4.3. Puntuación Final según Tipo de Query

Queries semánticas:

- Si score_semantico ≤ 0.25 : descartar documento
- Si score_semantico ≤ 0.75 : puntuación = (semántica $\times$ 40) + (léxico $\times$ 0.5)
- En otros casos: puntuación = (semántica $\times$ 30) + (léxico $\times$ 0.3)
- Final: puntuación $\times$ factor

Queries léxicas:

- Si score_léxico ≤ 3 y semántica ≤ 0.3 : descartar
- puntuación = score_léxico + (semántica $\times$ 10.0)
- Bonus multicampo (múltiples campos coincidentes): +3.0
- Final: puntuación $\times$ factor

3. Fuentes Bibliográficas

1. **Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990).** *Indexing by Latent Semantic Analysis*. Journal of the American Society for Information Science, 41(6), 391-407.
2. **Robertson, S., & Zaragoza, H. (2009).** *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333-389.
3. **Salton, G., Fox, E. A., & Wu, H. (1983).** *Extended Boolean Information Retrieval*. Communications of the ACM, 26(11), 1022-1036.
4. **Reimers, N., & Gurevych, I. (2019).** *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP, 3982-3992.
5. **Lewis, P., et al. (2020).** *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, 33, 9459-9474.
6. **Manning, C. D., Raghavan, P., & Schütze, H. (2008).** *Introduction to Information Retrieval*. Cambridge University Press.

4. Estadísticas del Corpus Recopilado

Métrica	Valor
Total de canciones únicas	10000
Canciones con información básica	10000 (100 %)
Canciones con género musical	10000 (100 %)
Canciones con tags	10000 (100 %)
Canciones con letra completa	10000 (100 %)
Tamaño total del corpus (texto)	12 MB

5. Arquitectura del Sistema

5.1. Flujo de Datos Completo

Fase 1: Indexación (una sola vez)

CSV Files → Cargar_datos() → Documentos unificados

Procesar_documentos()

Índice Invertido TF-IDF Embeddings Semánticos

Guardar_indice()

indice_musica.json + indice_musica.embeddings.npz

Fase 2: Búsqueda (cada consulta)

Usuario → Query → app.py (/buscar)

buscar_canciones_avanzado()

Procesar query Embedding query BM25 prep

Calcular scores (LSI + BM25 + Léxico)

Ranking y filtro (min_score)

¿Usar RAG? ¿Pocos resultados? ¿Buscar web?

Activación automática / Genius scraping

6. Módulos Implementados

6.1. Módulos Obligatorios

Módulo	Estado	Descripción
Adquisición de datos	Completo	Carga de CSVs y letras, scraping Genius
Indexación	Completo	Índice invertido, TF-IDF, embeddings
Modelo no básico SRI	Completo	LSI + BM25 + Booleano Extendido
Base de datos vectorial	Completo	Embeddings con FAISS-like (np.dot)
Módulo RAG	Completo	Pipeline RAG con Qwen2.5-3B
Posicionamiento	Completo	Ranking con scores, fragmentos destacados
Interfaz visual	Completo	Web con Flask, búsqueda dinámica
Búsqueda web	Completo	Scraping Genius con activación automática

6.2. Módulos Opcionales Implementados

Módulo	Estado	Justificación
Expansión y retroalimentación	Completo	Búsqueda web + caché de resultados
Evaluación	Completo	Métricas Precision@k, Recall@k, NDCG, MAP, MRR

7. Manual de Usuario

7.1. Inicio del Sistema

```
# 1. Clonar repositorio
git clone [repositorio]
cd proyecto_SRI

# 2. Construir y ejecutar con Docker (recomendado)
docker build -t sri-music-search .
docker run -p 5000:5000 sri-music-search

# 3. O ejecutar localmente
pip install -r requirements.txt
python app.py

# 4. Abrir navegador en http://localhost:5000
```

7.2. Interfaz de Usuario

La interfaz consta de:

- Campo de búsqueda con botón “Buscar”
- Checkbox para búsqueda web automática
- Área de respuesta RAG (análisis inteligente)
- Lista de resultados con título, artista, fragmento de letra, géneros y puntuación

7.3. Tipos de Consultas Soportadas

Tipo de consulta	Ejemplo	Comportamiento
Título exacto	“Bohemian Rhapsody”	Alta prioridad a coincidencia exacta
Artista	“Taylor Swift”	Muestra canciones del artista
Fragmento de letra	“we will rock you”	Búsqueda BM25 en letra
Tema/emoción	“canciones tristes sobre amor”	Dominancia semántica (LSI) + RAG

8. Estrategia de Posicionamiento

8.1. Factores de Ranking

Factor	Peso máximo	Descripción
Similitud semántica (LSI)	40 %	Coincidencia en espacio latente de temas
BM25 en letra	30 %	Relevancia probabilística en texto completo
Coincidencia título/artista	30 %	Ponderación por campo, bonificaciones exactas

8.2. Justificación de la Estrategia

Decisión	Justificación teórica	Evidencia empírica
Mayor peso a título	Modelo de campos ponderados (Salton, 1983)	Usuario recuerda título
Inclusión de LSI	LSI captura relaciones semánticas (Deerwester, 1990)	Permite “canciones relacionadas”
Normalización longitud	BM25 corrige sesgo por documentos largos	Canciones de 100-500 palabras
Umbral relevancia	Evita falsos positivos	Precisión óptima en resultados

9. Opinión Crítica

9.1. Bondades del Sistema

- **Modelo híbrido:** Integración de LSI + BM25 + Booleano
- **Multilingüismo:** Soporte para 5 idiomas con detección automática
- **RAG local:** Qwen2.5-3B ejecutándose localmente (sin API externa)
- **Búsqueda web automática:** Activación automática con umbral de resultados
- **Evaluación rigurosa:** Métricas estándar implementadas
- **Interfaz usable:** Flask + CSS moderno, responsive

9.2. Insuficiencias y Limitaciones

- Búsqueda semántica $O(n)$ - necesita FAISS
- Modelo Qwen2.5-3B en CPU (3-5s por respuesta)
- Sin caché de embeddings de consulta
- Precisión semántica limitada (modelo SBERT pequeño)
- Sin retroalimentación explícita

9.3. Trabajo Futuro

1. Implementar FAISS para búsqueda semántica $O(\log n)$
2. Agregar caché de respuestas para queries similares
3. Mejorar RAG con fine-tuning del modelo Qwen
4. Añadir recomendación colaborativa
5. Implementar feedback explícito (like/dislike)

10. Despliegue con Docker

Leer instructions.txt en la carpeta docker